	Fundamentos de Sistemas Embebidos
Facultad de Ingeniería	UNAM

Laboratorio de Fundamentos de Sistemas Embebidos

Profesor(a): Eduardo Fragoso Navarro

Asignatura: Fundamentos de Sistemas Embebidos

Grupo: 3

No de Práctica(s): 6

Integrante(s): Tepal Briseño Hansel Yael - 320078282

Ugartechea González Luis Antonio - 320223822

No. de brigada: 12

Semestre: 9°

Fecha de entrega: 13 de octubre del 2026

Observaciones:

CALIFICACIÓN: _____

Reconocimiento de Voz y de la Mano con IA

Objetivos

El objetivo de esta práctica fue integrar un sistema de captura distribuido en una tarjeta XIAO ESP32-S3 con dos aplicaciones de inteligencia artificial ejecutadas localmente en una computadora. En el primer ejercicio se recibió el audio del micrófono mediante UDP y se transcribió al español con `faster-whisper`. En el segundo se recibieron imágenes de la cámara y se detectaron los puntos de una mano mediante MediaPipe Hand Landmarker.

También se buscó comprender la separación de responsabilidades entre el dispositivo embebido y la computadora. El ESP32 realiza la adquisición, la detección básica de actividad y el envío de datos, mientras que la PC ejecuta los modelos de reconocimiento, que requieren más memoria y capacidad de cómputo.

Resultados y Análisis

Configuración del sistema

La comunicación se realizó en modo estación (STA). El ESP32 se conectó a la red WiFi de 2.4 GHz y utilizó la dirección fija `192.168.8.212`, correspondiente al equipo 12. La computadora tuvo que estar conectada a la misma red para poder intercambiar datagramas UDP con el puerto 3333. Se utilizó la misma estructura de firmware para los dos ejercicios y se seleccionó la aplicación activa desde `main.c`.

El proyecto se compiló para el target `esp32s3`. El binario de audio ocupó aproximadamente 751 kB y el binario de video 824 kB; ambos caben en la partición de aplicación de 1.5 MB. El proceso de carga verificó los hashes de los tres elementos escritos en la memoria flash y terminó con un reinicio automático de la tarjeta.

La práctica activa se seleccionó desde el archivo principal. Solo una de las dos funciones debe quedar definida para evitar ejecutar simultáneamente los dos periféricos:

```
// Audio o video: dejar solamente una linea activa
// #define PRACTICA sta_audio_ahorro_app_main
#define PRACTICA sta_video_ahorro_app_main

void app_main(void)
{
    PRACTICA();
}
```

Listing 1: Selección del ejercicio activo

Ejercicio 2: voz a texto

El programa del ESP32 muestrea el micrófono PDM a 16 kHz con muestras de 16 bits. Para reducir el consumo, el vigía permanece la mayor parte del tiempo en reposo y solo enciende el micrófono periódicamente. Cuando detecta ruido por tres mediciones consecutivas, comienza a enviar los bloques de audio que superan el umbral configurado. La computadora se registra mediante un datagrama `R` y recibe los bloques por UDP.

En `voz_a_texto.py` se acumulan 3 segundos de audio antes de llamar al modelo. Esto representa:

$$3\,\mathrm{s} \times 16000\,\frac{\text{muestras}}{\mathrm{s}} \times 2\,\frac{\text{bytes}}{\text{muestra}} = 96000\,\text{bytes}.$$

Se utilizó el modelo `base` de `faster-whisper`, ejecutado en CPU con `compute_type="int8"`. Antes de transcribir, las muestras enteras se convierten a valores de punto flotante en el intervalo aproximado $[-1, 1]$. La opción `vad_filter=True` permite ignorar los segmentos de silencio y el idioma se fija en español.

El bloque de transcripción empleado fue equivalente al siguiente:

```
audio = np.frombuffer(bloque, dtype=np.int16).astype(np.float32)
audio /= 32768.0
segmentos, _ = modelo.transcribe(
    audio, language="es", vad_filter=True)
for segmento in segmentos:
    print(segmento.text.strip(), flush=True)
```

Listing 2: Conversión y transcripción del audio

El modelo `tiny` tendría menor latencia y consumo de memoria, pero generalmente produciría más errores. `small` podría mejorar el reconocimiento, especialmente con ruido o palabras menos frecuentes, a cambio de mayor tiempo de procesamiento. El modelo `base` representa un compromiso razonable para una computadora de propósito general.

El uso de bloques de 3 segundos introduce una latencia mínima de ese orden. Si una palabra queda dividida entre dos bloques, puede perderse parte del contexto y la transcripción puede ser incorrecta o quedar incompleta. Una mejora sería usar bloques superpuestos o conservar una pequeña ventana del bloque anterior.

En el firmware, la tarea de audio solo envía un bloque cuando el nivel supera el umbral. La dirección de la computadora se aprende cuando el script envía el datagrama de registro:

```
recibidos = recvfrom(s_socket, mensaje, sizeof(mensaje) - 1,
                     0, (struct sockaddr *)&origen,
                     &largo_origen);
if (recibidos > 0) {
```

```
    s_pc_direccion = origen;
    s_pc_conocida = true;
}

if (s_pc_conocida && microfono_nivel() > NIVEL_UMBRAL) {
    sendto(s_socket, s_bloque, leidos, 0,
           (struct sockaddr *)&s_pc_direccion,
           sizeof(s_pc_direccion));
}
```

Listing 3: Registro de la PC y envío de audio por UDP

Ejercicio 3: detección de la mano

El firmware configura la cámara en escala de grises con una resolución de 96×96 píxeles. El programa de la PC convierte cada imagen recibida de BGR a RGB antes de entregarla a MediaPipe. El modelo Hand Landmarker se carga una sola vez desde `hand_landmarker.task` y se configura para buscar una mano.

Para cada mano detectada se dibujan sus 21 puntos y las conexiones definidas por MediaPipe. Además, se toman los puntos 4 y 8, correspondientes a la punta del pulgar y del índice, respectivamente. La distancia mostrada se calcula en píxeles mediante:

$$d = \sqrt{(x_8 - x_4)^2 + (y_8 - y_4)^2}.$$

La detección es más confiable cuando existe iluminación uniforme, la mano se encuentra dentro del encuadre y el fondo contrasta con ella. Puede fallar con poca luz, movimiento excesivo, oclusiones, fondos muy similares a la piel, distancias demasiado grandes o cuando aparecen varias manos, ya que el programa se configuró con `num_hands=1`. La cámara permanece apagada durante los intervalos de reposo; por ello la ventana de Python puede mostrar una imagen negra mientras el ESP32 no ha detectado movimiento o la PC no está en la misma red.

El procesamiento principal de cada imagen se realiza con la API Tasks de MediaPipe. Primero se convierte la imagen recibida a RGB, luego se ejecuta el detector y finalmente se dibujan las conexiones y la línea solicitada:

```
rgb = cv2.cvtColor(gris, cv2.COLOR_GRAY2RGB)
imagen = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb)
resultado = detector.detect(imagen)

foto = cv2.cvtColor(gris, cv2.COLOR_GRAY2BGR)
for mano in resultado.hand_landmarks:
    dibujar_mano(foto, mano)

pulgar = puntos[4]
```

```
indice = puntos[8]
distancia = math.hypot(indice[0] - pulgar[0],
                       indice[1] - pulgar[1])
cv2.line(foto, pulgar, indice, (0, 255, 255), 2)
```

Listing 4: Detección y dibujo de los puntos de la mano

En el ESP32 la cámara se inicializa en escala de grises y con dos buffers para mantener una imagen reciente disponible durante la transmisión:

```
camera_config_t cam_cfg = {
    .pixel_format = PIXFORMAT_GRAYSCALE,
    .frame_size = FRAMESIZE_96X96,
    .fb_count = 2,
    .fb_location = CAMERA_FB_IN_DRAM,
    .grab_mode = CAMERA_GRAB_LATEST,
};
ESP_ERROR_CHECK(esp_camera_init(&cam_cfg));
```

Listing 5: Configuración de la cámara del ESP32

Evidencias y problemas observados

El proceso de carga produjo una verificación correcta de los datos escritos en la memoria flash. Durante la prueba de video se observó inicialmente la ventana negra de espera. La causa no fue MediaPipe, sino que la computadora no tenía una ruta hacia `192.168.8.212`; estaba conectada a una red con dirección `10.x.x.x`. Al conectar la PC a la misma red WiFi del ESP32, el script pudo registrarse mediante UDP.

Para completar la evidencia solicitada por la práctica se deben agregar al directorio `reporte/img/` una captura de la terminal con una transcripción y una captura de la ventana de video con los puntos de la mano, y después insertarlas en esta sección. No se incluyeron imágenes inventadas porque dichas capturas no estaban disponibles en el directorio del reporte.

Conclusiones

El reconocimiento se ejecuta en la PC porque los modelos de Whisper y MediaPipe requieren más memoria, almacenamiento y capacidad de cómputo que la disponible en el ESP32 para esta implementación. Para ejecutar el reconocimiento directamente en el microcontrolador sería necesario usar modelos mucho más pequeños, cuantizados y optimizados, además de disponer de memoria suficiente y aceptar una menor precisión.

El procesamiento local tiene como ventajas que no requiere enviar audio o video a un servicio externo, puede funcionar sin Internet y ofrece mayor control sobre la privacidad. Sus desventajas son la necesidad de instalar los modelos, ocupar espacio en la computadora y disponer de suficiente

capacidad de procesamiento. Un servicio en la nube simplifica la ejecución y puede ofrecer modelos más grandes, pero depende de Internet, agrega latencia y expone los datos capturados a un tercero.

La arquitectura puede utilizarse en asistentes de voz locales, sistemas de accesibilidad, control de dispositivos mediante palabras, interfaces sin contacto, interpretación de señas, monitoreo de maquinaria y automatización doméstica. Para hacerla más inteligente se podrían agregar clasificación de palabras clave, identificación de hablantes, reducción de ruido, seguimiento temporal de la mano y un clasificador de gestos entrenado con los 21 puntos. Así sería posible reconocer comandos completos o traducir un conjunto de señas a acciones concretas.